VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Saksham Agrawal	24BCE10432
Slot	**Date of Submission**
D11	27/03/2026

1. Introduction

Most of the software that powers the modern world the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project something you can download, read about, and interact with on Linux and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional it is part of being a professional.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: Python

3. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots every page should contain original thinking and analysis written in your own words.

Part A Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Do not just copy the official description. Research the original context. What was frustrating or broken or inaccessible before this software existed? Why did the creator decide to build it and share it rather than keep it proprietary?

For example: Linus Torvalds was a student who wanted a free Unix-like system and could not afford one. Richard Stallman watched a printer malfunction and could not fix it because the source code was locked. The VLC team wanted to stream video over a university network but no tool existed that was free and open enough to adapt.

What is your software's origin story? Tell it properly.

Answer- Coding used to be a real headache. Before Python showed up, beginners had a hard time dealing with languages like C or C++, which were honestly just a mess of complicated rules. They weren't easy to read. They definitely weren't easy to write. Guido van Rossum hated that. He wanted something clear and simple. While he was working at CWI in the late 80s, he started building Python as a side project just for fun. He really just wanted to make programming more accessible. It worked. Instead of hiding his work, he put it out there for everyone to use and fix. That's how it grew. Now there's a huge global community because he decided to share it.

A2. The license what it actually says

Download or find the actual license text for your software. Read it. Then answer these questions in your report:

- What are the four freedoms of free software? Does your software's license grant all four?
- Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?
- What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?
- Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an example from your chosen software.

Answer- Python runs on the Python Software Foundation License. It's a permissive setup that lets you do pretty much whatever you want. You can run the code, poke around inside it, change things, or pass it along to someone else. It covers all the big bases of free software. If a company wants to tweak Python and sell their version without showing anyone their work, they can. The license doesn't force you to share. That’s the big difference with the GPL, which makes you show your modifications to everyone. The MIT license is even more relaxed and great for private projects. Personally, I'd go with MIT just to keep things flexible. People always talk about "free as in beer" versus "free as in freedom." The first one just means you didn't pay for it. The second is about having actual control. Python gives you both. You get the software for nothing and you're allowed to fix it however you see fit.

Think, don't copy	Answers copied from Wikipedia or AI tools will score zero. We are looking for your understanding and your words.

A3. The ethics of open source

Open source is not just a licensing model it is a statement about how knowledge and tools should be shared. Reflect on the following:

- Should all software be open source? Make an argument for and against.
- Is it ethical for a company to build a profitable product entirely on top of free community labor? (Think about how companies like Red Hat, Google, and Meta use Linux.)
- What does 'standing on the shoulders of giants' mean in the context of software? Does open source enable or undermine original innovation?

Answer- I don't think every piece of software needs to be open source. It’s great for learning and keeping things transparent, but sometimes you just have to keep things private for a business to actually work or to keep it secure. Companies making a profit off of stuff like Python is fine as long as they're actually helping out and giving back. It feels wrong when a giant corporation just takes and takes without ever chipping in. Everyone talks about standing on the shoulders of giants. It basically means you aren't reinventing the wheel, just like how Python was built using ideas from languages that came before it. This setup is a huge win for innovation because you can just grab a solid idea and make it better. Maybe it stops people from being 100 percent original every now and then. That’s a small price to pay for not starting from zero every time.

There are no correct answers here. We are grading the depth of your thinking, not your conclusion.

Part B Linux Footprint (Unit 2)

Research and document how your chosen software lives on a Linux system. Where does it install? What user does it run as? How is it managed? This section requires you to actually install the software on Linux either on your machine, a virtual machine, or a lab system.

- Installation method is it available via RPM, apt, or must it be compiled from source?
- Key directories where are its binaries, config files, and logs stored? (e.g., /usr/bin, /etc, /var/log)
- User and group what Linux user and group does it run under? Why does this matter for security?
- Service management can it be started, stopped, or restarted as a service? Show the commands.
- Update model how does the open-source community push security patches to your machine?

Answer- Most Linux systems come with Python already tucked away inside, but you can always grab it yourself using apt install python3 if you're on Ubuntu or yum for the RPM side of things. It's pretty straightforward. You'll find the actual binaries sitting in /usr/bin while the libraries stay in /usr/lib. You won't really need to mess with configuration files. Python just runs under your own user account instead of some weird, dedicated service profile. This makes things flexible, though you've got to watch your permissions so you don't break anything. It isn't a background service. Because of that, you aren't going to manage it with systemctl. Some apps written in Python might run as services, but the language itself doesn't. When it comes to security or new features, the open-source community and the folks maintaining the packages handle the heavy lifting. You just run your normal system updates to get the latest patches they've released. It's a solid system that usually just works.

Part C The FOSS Ecosystem (Units 3 & 4)

No open-source project exists in isolation. Map the ecosystem around your chosen software.

- What other open-source tools does it depend on? (Libraries, compilers, runtimes)
- What has it enabled or inspired? (Forks, spin-offs, tools built on top of it)
- Where does it fit in the LAMP stack, or how does it relate to the web?
- Is there a community mailing lists, forums, conferences, IRC channels? Who governs it?

Answer- Python relies on a few heavy hitters like GCC to handle compilation and OpenSSL to keep things secure. You'll also need libraries like libc to get everything running smoothly. It's pretty much the backbone for massive frameworks like Django or Flask, and it's basically taken over the world of AI and data science. People use it for automation all the time too. In the web world, it's a go-to backend language. It often steps in to replace PHP in the old LAMP stack by handling APIs and complex frameworks. The community is huge. You'll find people helping each other out on mailing lists, forums, or at big meetups like PyCon. It's all kept in check by the Python Software Foundation. They're the ones managing the development and setting the standards for how the whole thing works. It's a solid setup.

Part D Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source	Proprietary Alternative
Cost	Free to use, no licensing fees	Expensive license required
Security (who can audit the code?)	Source code is open; anyone can audit and find issues	Closed source; only company can audit
Support and reliability	Community support, forums, documentation	Official support, dedicated customer service
Freedom to modify	Full freedom to modify and redistribute	No access to source code; cannot modify

Community vs corporate control	Driven by global community and python software foundation	Controlled and managed by a single company (MathWorks)
Your overall verdict	Flexible, cost-effective, and widely used for many applications	Reliable and well-supported but expensive and less flexible

Fill in the table as part of your report, then write a two-paragraph conclusion: given everything you have learned, would you recommend your chosen software for a real-world deployment? Would you contribute to it?

Answer- Based on the table we just filled out, here are the two concluding paragraphs for your report. I've shifted the tone to be a bit more personal and direct, like we did for the earlier sections.

I'd definitely recommend Python for any real-world deployment, especially in fields like cybersecurity or data science. It is hard to beat a tool that gives you access to thousands of free libraries and the ability to scale up on cloud servers without paying a single cent in licensing fees. Most modern tech stacks are built on Python for a reason. It has basically become the industry standard at this point. This makes finding talent to maintain your code much easier than it would be with some niche proprietary tool that requires expensive certifications.

As for contributing, I am actually planning to look into the documentation side of things first. I might not be ready to rewrite the core C code just yet, but helping to clarify the guides for new students at VIT or elsewhere is a great way to give back. It is a cool feeling knowing that even a small fix can help millions of other developers. The open-source world only works if people actually participate. I'd really like to be a part of that cycle instead of just using the tools for free.

4. Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software

# --- Variables ---
STUDENT_NAME=""          # Fill in your name
SOFTWARE_CHOICE=""       # Fill in your chosen software

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)

# --- Display ---
echo "================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
# TODO: Add distro name, date, and license message
```

Output-

```
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ nano script1.sh
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ ./script1.sh
==============================
 Open Source Audit — Saksham Agrawal
==============================
Software : Python
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : lenovo
Uptime   : up 14 minutes
Date     : Thu Mar 26 15:28:30 UTC 2026
Distro   : Ubuntu 24.04.4 LTS
License  : GNU/Linux is mostly GPL licensed
```

Script 2 FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE=""   # e.g. httpd, mysql, vlc, firefox

# Check if package is installed
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

# TODO: Add a case statement that prints a one-line
# philosophy note about the package based on its name
case $PACKAGE in
    httpd)  echo "Apache: the web server that built the open internet" ;;
    mysql)  echo "MySQL: open source at the heart of millions of apps" ;;
    # TODO: Add your software and 3 others
esac
```

Output-

```
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ nano script2.sh
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ ./script2.sh
python3 is installed.
ii  libpython3-stdlib:amd64       3.12.3-0ubuntu2.1                    amd64        interactive high-level object-oriented language (default python3 version)
ii  libpython3.12-minimal:amd64   3.12.3-1ubuntu0.11                   amd64        Minimal subset of the Python language (version 3.12)
ii  libpython3.12-stdlib:amd64    3.12.3-1ubuntu0.11                   amd64        Interactive high-level object-oriented language (standard library, version 3.12)
ii  libpython3.12t64:amd64        3.12.3-1ubuntu0.11                   amd64        Shared Python runtime library (version 3.12)
ii  python3                       3.12.3-0ubuntu2.1                    amd64        interactive high-level object-oriented language (default python3 version)
ii  python3-apport                2.28.1-0ubuntu3.8                    all          Python 3 library for Apport crash report handling
ii  python3-apt                   2.7.7ubuntu5.2                       amd64        Python 3 interface to libapt-pkg
ii  python3-attr                  23.2.0-2                             all          Attributes without boilerplate (Python 3)
ii  python3-automat               22.10.0-2                            all          Self-service finite-state machines for the programmer on the go
ii  python3-babel                 2.10.3-3build1                       all          tools for internationalizing Python applications - Python 3.x
ii  python3-bcrypt                3.2.2-1build1                        amd64        password hashing library for Python 3
ii  python3-blinker               1.7.0-1                              all          Fast, simple object-to-object and broadcast signaling (Python3)
ii  python3-certifi               2023.11.17-1                         all          root certificates for validating SSL certs and verifying TLS hosts (python3)
ii  python3-cffi-backend:amd64    1.16.0-2build1                       amd64        Foreign Function Interface for Python 3 calling C code - runtime
ii  python3-chardet               5.2.0+dfsg-1                         all          Universal Character Encoding Detector (Python3)
ii  python3-click                 8.1.6-2                              all          Wrapper around optparse for command line utilities - Python 3.x
ii  python3-colorama              0.4.6-4                              all          Cross-platform colored terminal text in Python - Python 3.x
ii  python3-commandnotfound       23.04.0                              all          Python 3 bindings for command-not-found.
ii  python3-configobj             5.0.8-3                              all          simple but powerful config file reader and writer for Python 3
ii  python3-constantly            23.10.4-1                            all          Symbolic constants in Python
ii  python3-cryptography          41.0.7-4ubuntu0.1                    amd64        Python library exposing cryptographic recipes and primitives (Python 3)
ii  python3-dbus                  1.3.2-5build3                        amd64        simple interprocess messaging system (Python 3 interface)
ii  python3-debconf               1.5.86ubuntu1                        all          interact with debconf from Python 3
ii  python3-distro                1.9.0-1                              all          Linux OS platform information API
ii  python3-distro-info           1.7build1                            all          information about distributions' releases (Python 3 module)
ii  python3-distupgrade           1:24.04.28                           all          manage release upgrades
ii  python3-gdbm:amd64            3.12.3-0ubuntu1                      amd64        GNU dbm database support for Python 3.x
ii  python3-gi                    3.48.2-1                             amd64        Python 3 bindings for gobject-introspection libraries
ii  python3-hamcrest              2.1.0-1                              all          Hamcrest framework for matcher objects (Python 3)
ii  python3-httplib2              0.20.4-3                             all          comprehensive HTTP client library written for Python3
ii  python3-hyperlink             21.0.0-5                             all          Immutable, Pythonic, correct URLs.
ii  python3-idna                  3.6-2ubuntu0.1                       all          Python IDNA2008 (RFC 5891) handling (Python 3)
ii  python3-incremental           22.10.0-1                            all          Library for versioning Python projects
ii  python3-jinja2                3.1.2-1ubuntu1.3                     all          small but fast and easy to use stand-alone template engine
ii  python3-json-pointer          2.0-0ubuntu1                         all          resolve JSON pointers - Python 3.x
ii  python3-jsonpatch             1.32-3                               all          library to apply JSON patches - Python 3.x
ii  python3-jsonschema            4.10.3-2ubuntu1                      all          An(other) implementation of JSON Schema (Draft 3, 4, 6, 7)
ii  python3-jwt                   2.7.0-1                              all          Python 3 implementation of JSON Web Token
ii  python3-launchpadlib          1.11.0-6                             all          Launchpad web services client library (Python 3)
ii  python3-lazr.restfulclient    0.14.6-1                             all          client for lazr.restful-based web services (Python 3)
ii  python3-lazr.uri              1.0.6-3                              all          library for parsing, manipulating, and generating URIs
ii  python3-markdown-it           3.0.0-2                              all          Python port of markdown-it and some its associated plugins
ii  python3-markupsafe            2.1.5-1build2                        amd64        HTML/XHTML/XML string library
ii  python3-mdurl                 0.1.2-1                              all          Python port of the JavaScript mdurl package
ii  python3-minimal               3.12.3-0ubuntu2.1                    amd64        minimal subset of the Python language (default python3 version)
ii  python3-netifaces:amd64       0.11.0-2build3                       amd64        portable network interface information - Python 3.x
ii  python3-netplan               1.1.2-8ubuntu1~24.04.1               amd64        Declarative network configuration Python bindings
ii  python3-newt:amd64            0.52.24-2ubuntu2                     amd64        NEWT module for Python3
ii  python3-oauthlib              3.2.2-1                              all          generic, spec-compliant implementation of OAuth for Python3
ii  python3-openssl               23.2.0-1                             all          Python 3 wrapper around the OpenSSL library
ii  python3-pkg-resources         68.1.2-2ubuntu1.2                    all          Package Discovery and Resource Access using pkg_resources
ii  python3-problem-report        2.28.1-0ubuntu3.8                    all          Python 3 library to handle problem reports
ii  python3-pyasn1                0.4.8-4ubuntu0.1                     all          ASN.1 library for Python (Python 3 module)
ii  python3-pyasn1-modules        0.2.8-1                              all          Collection of protocols modules written in ASN.1 language (Python 3)
ii  python3-pycurl                7.45.3-1build2                       amd64        Python bindings to libcurl (Python 3)
ii  python3-pygments              2.17.2+dfsg-1                        all          syntax highlighting package written in Python 3
ii  python3-pyparsing             3.1.1-1                              all          alternative to creating and executing simple grammars - Python 3.x
ii  python3-pyrsistent:amd64      0.20.0-1build2                       amd64        persistent/functional/immutable data structures for Python
ii  python3-requests              2.31.0+dfsg-1ubuntu1.1               all          elegant and simple HTTP library for Python3, built for human beings
ii  python3-rich                  13.7.1-1                             all          render rich text, tables, progress bars, syntax highlighting, markdown and more
ii  python3-serial                3.5-2                                all          pyserial - module encapsulating access for the serial port
ii  python3-service-identity      24.1.0-1                             all          Service identity verification for pyOpenSSL (Python 3 module)
ii  python3-setuptools            68.1.2-2ubuntu1.2                    all          Python3 Distutils Enhancements
ii  python3-six                   1.16.0-4                             all          Python 2 and 3 compatibility library
ii  python3-software-properties   0.99.49.3                            all          manage the repositories that you install software from
ii  python3-systemd               235-1build4                          amd64        Python 3 bindings for systemd
ii  python3-twisted               24.3.0-1ubuntu0.1                    all          Event-based framework for internet applications
ii  python3-typing-extensions     4.10.0-1                             all          Backported and Experimental Type Hints for Python
ii  python3-tz                    2024.1-2                             all          Python3 version of the Olson timezone database
ii  python3-update-manager        1:24.04.12                           all          Python 3.x module for update-manager
ii  python3-urllib3               2.0.7-1ubuntu0.6                     all          HTTP library with thread-safe connection pooling for Python3
ii  python3-wadllib               1.3.6-5                              all          Python 3 library for navigating WADL files
ii  python3-yaml                  6.0.1-2build2                        amd64        YAML parser and emitter for Python3
ii  python3-zope.interface        6.1-1build1                          amd64        Interfaces for Python3
ii  python3.12                    3.12.3-1ubuntu0.11                   amd64        Interactive high-level object-oriented language (version 3.12)
ii  python3.12-minimal            3.12.3-1ubuntu0.11                   amd64        Minimal subset of the Python language (version 3.12)
Python: simple and powerful programming language
```

Script 3 Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Starter structure

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "-------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
```

```
        echo "$DIR does not exist on this system"
    fi
done

# TODO: Add a section that checks if your software's
# config directory exists and prints its permissions
```

Output-

```
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ nano script3.sh
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 17M
/home => Permissions: drwxr-xr-x root root | Size: 36K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 20K
```

Script 4 Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages

LOGFILE=$1
KEYWORD=${2:-"error"}     # Default keyword is 'error'
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# TODO: Add a do-while style retry if the file is empty,
# and print the last 5 matching lines using tail + grep
```

Output-

```
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ nano script4.sh
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ ./script4.sh /var/log/syslog error
Keyword 'error' found 7 times in /var/log/syslog
```

Script 5 The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# TODO: Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Output-

```
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ nano script5.sh
lenovo@SakshamLOQ:/mnt/c/Users/LENOVO$ ./script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use: Python
2. In one word, what does freedom mean to you? Innovation
3. What would you build and share? AItools

Manifesto saved to manifesto.txt
Your manifesto:
On Thu Mar 26 15:37:50 UTC 2026, I believe Python represents Innovation. I want to build AItools and share it freely with everyone.
```

5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1–2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3–4 working*
7	**Ecosystem**	Write Part C FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

6. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story depth, original research, storytelling	12	6	*Part A1*
2	License analysis accuracy and understanding of freedoms	12	6	*Part A2*
3	Ethical reflection quality of argument and original thinking	10	5	*Part A3*
4	Linux footprint technical accuracy and screenshots	10	5	*Part B*
5	Ecosystem and LAMP connection	8	4	*Part C*
6	Open source vs proprietary comparison	8	4	*Part D*

Shell Scripts (40 marks 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness does it run without errors?	4	*Script executes on Linux and produces expected output*
Concepts does it use the right shell constructs?	2	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	2	*Every non-obvious line has a comment; script purpose is clear*

Academic integrity	All written sections must be in your own words. Code that is copy-pasted from the internet without understanding will be zero-marked. Running a script and explaining what it does in a conversation is considered a passing demonstration of understanding.

7. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab the LICENSE file is always there

A closing thought

Every tool you will use in your career the editor, the compiler, the server, the database was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

8. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?
GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.